AWK — Reference

An **AWK** subset compiled with `lex` + `yacc` + `cc` (`awk.l` / `awk.y` lower AWK to C; `cc` lowers the C to .NET IL). AWK is a pattern-action language for text: a program is a list of `pattern { action }` rules, and the runtime reads input a record (line) at a time, splits it into fields, and runs every rule whose pattern matches. Values are dynamically typed — a value is a string that is coerced to a number when used arithmetically — so every value is represented as a `char*` with on-demand numeric coercion. Arrays are string-keyed hash maps. User functions compile to `public static` methods on `CProgram`, so they are callable from C#/VB.NET (Activity 10).

```
awk prog.awk                # compile to prog.exe; run as: prog.exe < input
awk prog.awk -o app.exe     # choose the output name
awk lib.awk --dll           # compile a file of functions to a library for C#/VB.NET
```

> This AWK got its own milestone: the LALR generator was made ~30× faster first, because AWK's expression grammar (with string concatenation by juxtaposition) is the textbook hard case — it produces ~540 shift/reduce conflicts, all resolved correctly by the declared operator precedence.

Quick Reference

```
# comment
BEGIN { ... }               # runs once before input
pattern { action }          # action runs for each record matching pattern
END   { ... }               # runs once after input

$1                          # first field; $0 whole record; NF field count; NR record no.
x = "a" y                   # juxtaposition concatenates ("a" then the value of y)
x += 1   x++   x %= 3       # compound assignment, increment
$2 > 100                    # a relational pattern
/regex/                     # a regex pattern (matches against $0)
$1 ~ /^[0-9]/               # explicit match;  !~ is non-match

if (c) ... else ...         while (c) ...        do ... while (c)
for (i = 1; i ≤ n; i++) ...  for (k in arr) ...
count[$1]++                 # associative array
function name(a, b) { return a + b }
```

Data types

Dynamic: a value is a **string** that is **a number when it looks like one**. `"3" + 4` is `7`; `"3" "4"` (juxtaposition) is `"34"`. The only values are scalars (string/number) and **arrays** (string-keyed, associative).

Uninitialised variables are the empty string / zero. Truth: a value is true if it is a non-zero number or a non-empty non-numeric string.

Patterns and records

Each input line is a **record**, split into **fields** `$1, $2, …` on the field separator `FS` (default whitespace); `$0` is the whole record, `NF` the field count, `NR` the record number. A rule's pattern may be `BEGIN` / `END`, a boolean expression (`$3 > 0`), or a regular expression (`/re/` , matched against `$0`). A rule with no pattern runs for every record; a pattern with no action prints `$0` .

Statements / Commands

`print` , `printf` , assignment (and `+= -= *= /= %=` , `++` , `--`), `if/else` , `while` , `do/while` , `for(;;)` , `for(k in a)` , `delete a[k]` , `next` , `exit` , `return` , `break` , `continue` , and `{ … }` blocks.

Functions

User functions: `function name(params) { … }` . Parameters are passed by value (arrays by reference is outside this subset). Built-ins included: `length` , `substr` , `index` , `split` , `sprintf` , `toupper` , `tolower` , `int` , `sqrt` , `sin` , `cos` , `exp` , `log` , `atan2` . Each user function compiles to `char* f_<name>(…)` on `CProgram` .

Input / Output

`print a, b` writes the arguments separated by `OFS` and terminated by `ORS` (a newline); `printf fmt, …` does C-style formatting (`%d %s %f %g %x %c …` , up to eight arguments). Input is read from standard input, record by record.

Graphics

None — AWK is a text-processing language; Activity 8 draws a text histogram.

Notes

- Fields and arrays use `$` / `[]` ; arrays are 0 builtin-indexed by their string keys.
- `~` / `!~` are regex match / non-match; comparison is numeric when both sides look numeric, else string (so `0 < "cats"` is a *string* comparison — as in real AWK).
- Concatenation is **juxtaposition** (no operator): `"x" y z` .
- The regex engine supports `. * + ? [ ] ^ $ \` and literal characters (no alternation `|` or groups `( )`).

Subset boundaries

A broad, working core. Not included: `sub` / `gsub` / `match` (in-place regex substitution and `RSTART` / `RLENGTH`), `getline`, multiple input files / `FILENAME` / `FNR`, output redirection (`print > "file"`, `|` pipes), `printf` beyond eight arguments, passing arrays to user functions, regex alternation/grouping, and the `RS` / `SUBSEP` / `CONVFMT` knobs. Numbers are IEEE doubles.

Tutorial

Every example was compiled and run with `awk`; the output shown is real.

1. Your first program

```
BEGIN { print "hello from AWK" }
```

```
hello from AWK
```

`BEGIN` runs once before any input. Without a `BEGIN` / `END`, a rule runs per input line.

2. Variables and data types

```
BEGIN {
    x = 6
    print x * 7
    print "3" + 4          # numeric coercion
    print "v" 42           # juxtaposition → concatenation
}
```

```
42
7
v42
```

`"3" + 4` coerces the string to a number; `"v" 42` concatenates by juxtaposition.

3. Flow control

```
BEGIN {
    for (i = 1; i ≤ 5; i++) {
        if (i % 3 == 0) print "fizz"
        else print i
    }
}
```

```
1
2
fizz
4
5
```

4. Fields and records — the heart of AWK

```
{ sum += $1 }                 # $1 is the first field
END { print "total:", sum, "rows:", NR }
```

Input:

```
10
20
30
```

Output:

```
total: 60 rows: 3
```

The action runs per line; `$1` is the first field, `NR` the record count, and `sum` starts empty (zero).

5. Functions

```
function sq(n)  { return n * n }
function fib(n) { if (n < 2) return n; return fib(n - 1) + fib(n - 2) }
BEGIN { print sq(7), fib(15) }
```

```
49 610
```

6. Memory management

There is nothing to free — storage is managed by the **.NET garbage collector**:

- Strings, numbers, and array entries are heap objects created on demand.
- Associative arrays grow as keys are added; `delete a[k]` removes one.
- The CLR reclaims everything unreachable; AWK has no manual deallocation.

7. Strings

```
BEGIN {
    s = "hello"
    print length(s), toupper(s), substr(s, 2, 3)
    n = split("a:b:c", part, ":")
    print n, part[1], part[3]
}
```

```
5 HELLO ell
3 a c
```

`length`, `toupper`, `substr`, and `split` (which fills an array and returns the count) are built in.

8. Drawing a picture — a text histogram

```
{ bar = ""; for (i = 0; i < $2; i++) bar = bar "#"; print $1, bar }
```

Input:

```
cats 3
dogs 6
birds 2
```

Output:

```
cats ###
dogs ######
birds ##
```

Each row builds a bar of `$2` hashes by repeated concatenation.

9. Associative arrays — word frequencies

The classic AWK one-liner:

```
{ for (i = 1; i ≤ NF; i++) freq[$i]++ }
END { for (w in freq) print w, freq[w] }
```

Input:

```
the cat the dog
the cat
```

Output (hash order):

```
the 3
dog 1
cat 2
```

`freq` is keyed by the words themselves; `for (w in freq)` iterates the keys.

10. Calling AWK from C# / VB.NET

Compile a file of functions as a library. AWK values are strings (`char*`), so C# pushes arguments into the runtime with `CRuntime.InternString` and reads results back out, after a one-time `awk_init()`:

```
# lib.awk
function greet(who) { return "hello, " who "!" }
function fib(n)     { if (n < 2) return n; return fib(n - 1) + fib(n - 2) }
```

```
awk lib.awk --dll          # → awklib.dll
```

```
using CRuntimeLib;

CProgram.awk_init();
int g = CProgram.f_greet(CRuntime.InternString("Ada"));
int f = CProgram.f_fib(CRuntime.InternString("15"));
// read the char* results out of the arena (strlen + LdU8) ...
Console.WriteLine($"greet = {CStr(g)}, fib(15) = {CStr(f)}");
```

```
greet = hello, Ada!, fib(15) = 610
```

C# is calling real AWK functions. The interop is at the value level (string handles + `InternString`) — the honest consequence of AWK's stringly-typed model. From here the natural extensions are `sub` / `gsub`, `getline`, and multiple input files (*Subset boundaries*).